PROJET DE FIN DE MODULE · AGENT LLM

# Concert Agent

Trouver de vrais concerts avec un agent LLM qui utilise des tools.

- Projet Python / Flask
- SQLite
- Agent LLM avec confirmation
- Connexion Spotify
- Démonstration en 3 scénarios

**NOTES ORAL**

Phrase d'ouverture : « un assistant qui ne se contente pas de répondre : il cherche, filtre, propose et enregistre — avec validation humaine. »

# Le problème

Un chatbot seul peut inventer une date, une salle ou un prix.

Pour un concert, une information inventée est inutilisable :

- mauvaise date ;
- mauvaise ville ;
- prix faux ;
- aucune trace de ce qui a été fait.

**Objectif :** un agent fiable, traçable et contrôlable.

NOTES ORAL

**Notes :** exemple simple : « Je veux voir Gims demain à Paris » ne doit pas produire une fausse date.

# La solution

Un agent, ce n'est pas seulement un modèle qui parle : c'est un modèle qui peut **choisir des actions**.

1. **Agent LLM** : comprend la demande en français.
2. **Tools Python** : cherchent réellement les concerts.
3. **SQLite** : garde les préférences, favoris, historique et traces.
4. **Confirmation** : aucune écriture sans accord explicite.
5. **Spotify** : importe les artistes réellement écoutés.

**NOTES ORAL**

**Notes :** analogie : le LLM est le cerveau, les tools sont les mains, SQLite est la mémoire, la confirmation est le garde-fou.

# Démo 1 : recherche avec préférences

## Demande

> Trouve un concert de mes artistes préférés en Auvergne-Rhône-Alpes le mois prochain.

## Ce que fait l'agent

- `consulter_preferences` ;
- `rechercher_concerts` avec la région et la période ;
- recherche large sur tous les artistes suivis, jusqu'à 40 ;
- réponse construite à partir des résultats Ticketmaster.

## Résultat observé

- Gims — LDLC Arena, Décines ;
- 02/11/2026 et 03/11/2026.

Chaque concert affiche : date, heure, lieu, prix et lien de billetterie.

**NOTES ORAL**

**Notes :** montrer les traces. Si Ticketmaster ne trouve rien, la démo est signalée et les données ne sont pas présentées comme réelles.

# Démo 2 : aucun résultat

## Demande

> Je veux voir un concert à Paris demain, max 5 €.

## Ce que fait l'agent

- recherche réelle avec les filtres ;
- aucun concert ne correspond ;
- réponse : « Je ne peux pas inventer de dates ni de salles. »

## Point clé

L'absence de résultat est une **vraie réponse**, pas un échec.

NOTES ORAL

**Notes :** c'est la preuve que le modèle n'est pas seul ; les données viennent du tool.

# Démo 3 : écriture avec confirmation

## Demande

> Ajoute le concert de Gims du 2 novembre 2026 à Décines à mes favoris.

## Séquence

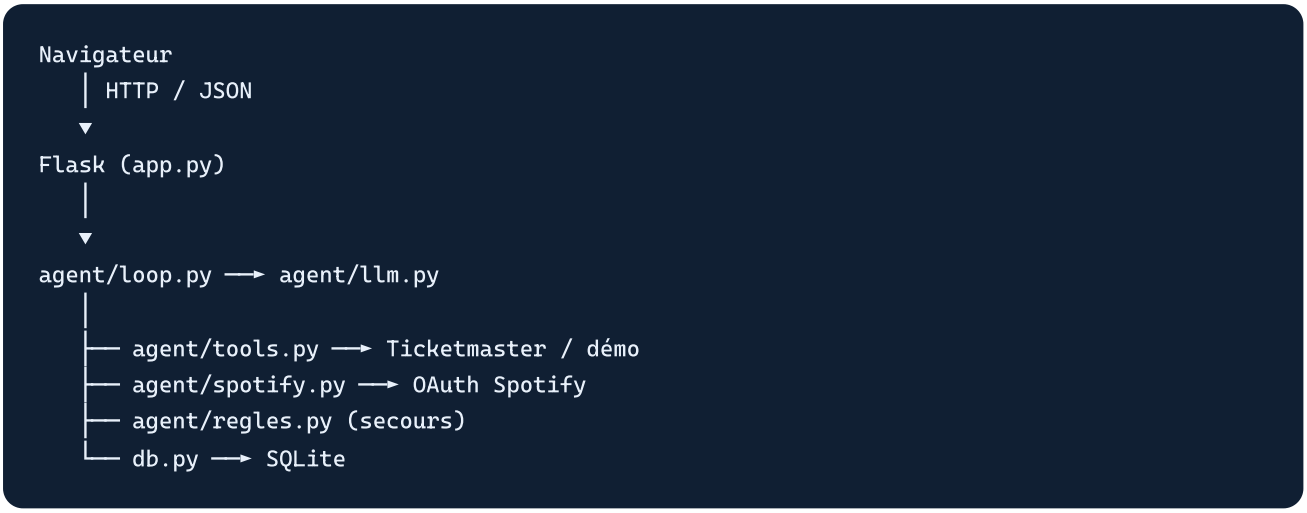
1. L'agent prépare l'action.
2. Il demande : « Je vais ajouter Gims — 02/11/2026 à Décines (LDLC Arena). Confirmer ? »
3. L'utilisateur clique sur **Oui**.
4. Le tool `ajouter_favori` écrit dans SQLite.
5. Le favori apparaît dans la barre latérale.

**Variante :** cliquer sur **Non** et vérifier qu'aucun favori n'est ajouté.

NOTES ORAL

**Notes :** insist : la confirmation est imposée par le code Python, pas seulement par le prompt.

# Architecture




- Une seule porte d'entrée pour le LLM.
- Une seule porte d'entrée pour les tools.
- Une seule base pour les données et les traces.

**NOTES ORAL**

**Notes :** cette séparation rend le projet testable et remplaçable (Gemini, OpenAI, Ollama).

# Les tools

| Tool | Type |
|---|---|
| `consulter_preferences` | lecture |
| `rechercher_concerts` | lecture |
| `lister_favoris` | lecture |
| `historique_agent` | lecture |
| `ajouter_favori` | écriture |
| `ajouter_artiste_suivi` | écriture |

**4 tools de lecture, 2 tools d'écriture.**

Chaque tool a une description en français : c'est le mode d'emploi du LLM.

NOTES ORAL

**Notes :** le LLM ne voit pas le code Python ; il voit le nom, la description et le schéma JSON des arguments.

# SQLite et traçabilité

SQLite garde :

- les préférences et les artistes suivis ;
- les favoris ;
- les interactions ;
- les traces des tools ;
- les actions en attente de confirmation.

Chaque appel de tool laisse une trace : nom, arguments, statut, durée.

Chaque favori est cliquable : sa fiche réunit date, heure, salle, ville, région, prix, source et lien de billetterie.

**NOTES ORAL**

**Notes :** montrer le panneau « Traces » dans le chat, puis ouvrir la fiche d'un favori.

# Confirmation et sécurité

- Les tools d'écriture sont marqués `is_write=True`.
- L'action est stockée dans `pending_actions`.
- « Oui » exécute ; « Non » annule.
- `ajouter_favori` vérifie que le concert vient de la dernière recherche.
- Les secrets sont dans `.env`, hors Git.
- Les jetons Spotify sont en base, pas dans le cookie ; le `state` OAuth est vérifié.

**NOTES ORAL**

**Notes :** la confirmation est une barrière technique, pas une phrase dans le prompt.

# Tests

```
.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Résultat :

```
Ran 15 tests ... OK
```

Les tests couvrent les 3 scénarios, la confirmation, le refus, l'historique, les préférences, les tools et Spotify sans appel réseau.

NOTES ORAL

**Notes :** les tests utilisent une base temporaire. Ils ne remplacent pas un essai réel de Ticketmaster et Spotify.

# Limites et perspectives

## Ce qui est livré

- concerts réels via Ticketmaster ( `locale=fr-fr` ) ;
- recherche large plafonnée (40 artistes, 30 résultats) ;
- LLM réel validé avec `gemini-3.5-flash-lite` ;
- mode règles en secours ;
- Spotify réel, limité à 5 comptes en développement ;
- données de démonstration clairement signalées.

## Ce qui reste à faire

- élargir la couverture concerts ;
- notifications et partage de favoris ;
- disque persistant pour Render.

NOTES ORAL

**Notes :** annoncer honnêtement ce qui est démontré et ce qui reste à brancher.

# Conclusion

**Concert Agent** montre qu'un LLM peut être utile sans être cru sur parole :

- il choisit ses tools ;
- Python exécute ;
- Spotify fournit les artistes de l'utilisateur ;
- SQLite garde la trace ;
- l'utilisateur confirme les écritures.

NOTES ORAL

**Notes :** phrase de fin : « L'agent ne devine pas les concerts : il les cherche, et il demande la permission avant d'agir. »